Information Retrieval

In A Nutshell

Table of Contents

1 IR Overview

2 Indexing

3 Boolean Retrieval Model

4 Vector Space Model (VSM)

5 Term weighting with $tf \times idf$

6 Further Models

7 Conclusion

Credits

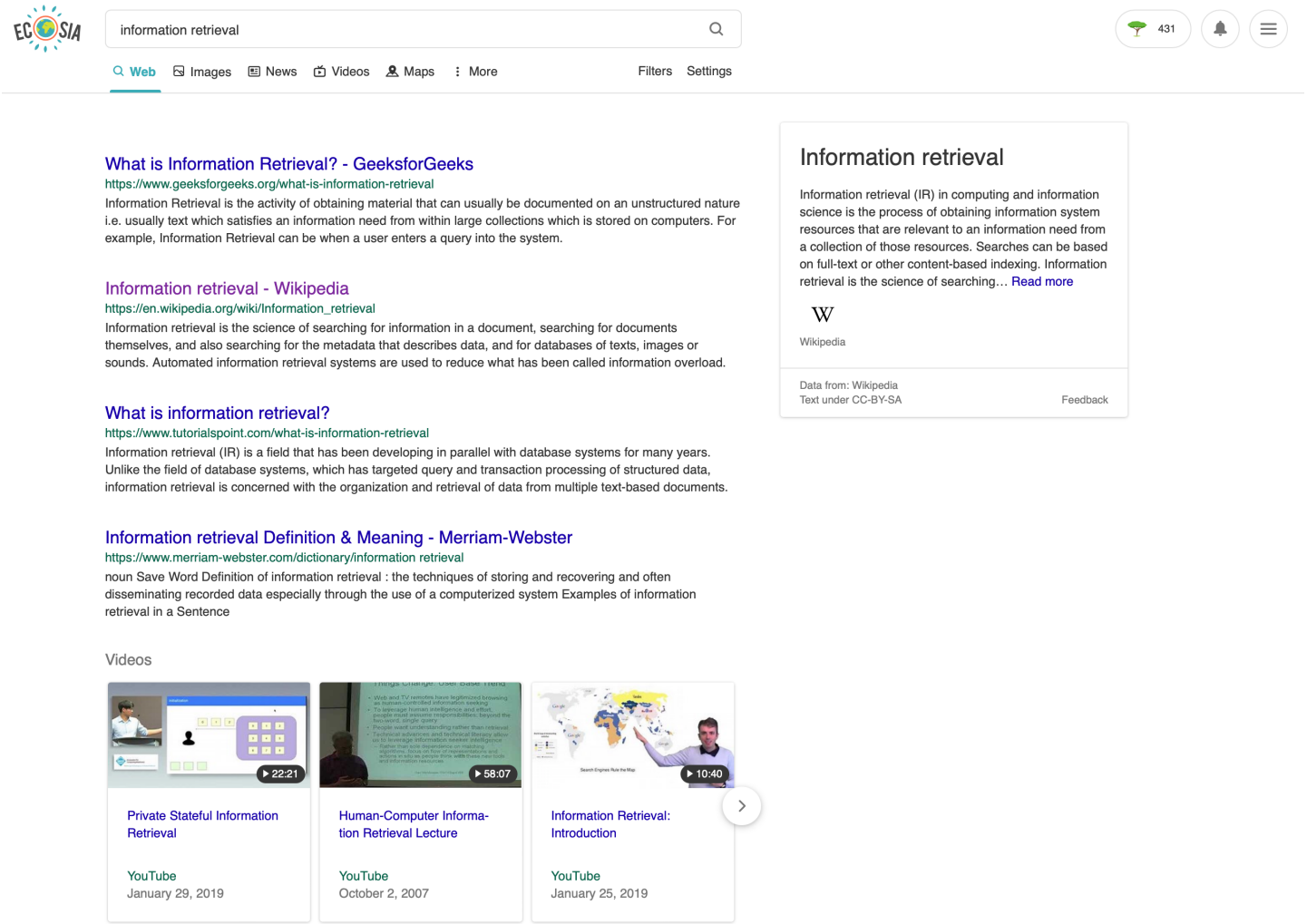
Thanks to Ingo Frommholz for the slides.

Some slides, figures and bullet points are borrowed from Baeza-Yates & Ribeiro-Neto, Modern Information Retrieval, 2nd Edition `http://www.mir2ed.org/`.

Others are taken from the Information Retrieval module taught at Queen Mary, University of London (thanks to Thomas Roelleke and Hany Azzam).

IR Overview

You all know this...



IR Applications

- Popular examples are Internet search engines, like Google, Bing, etc.
  - You also find more local searches, e.g. searching certain Web sites (like YouTube, our university Web site,...)
- Classically, IR was about searching in literature databases and library catalogues
- Other examples include:
  - Desktop Search (e.g. Spotlight in MacOS)
  - Enterprise Search
  - Expert Search
  - Retrieval-augmented Generation (with LLMs)
- IR and search are important parts in the whole Information Seeking & Searching process

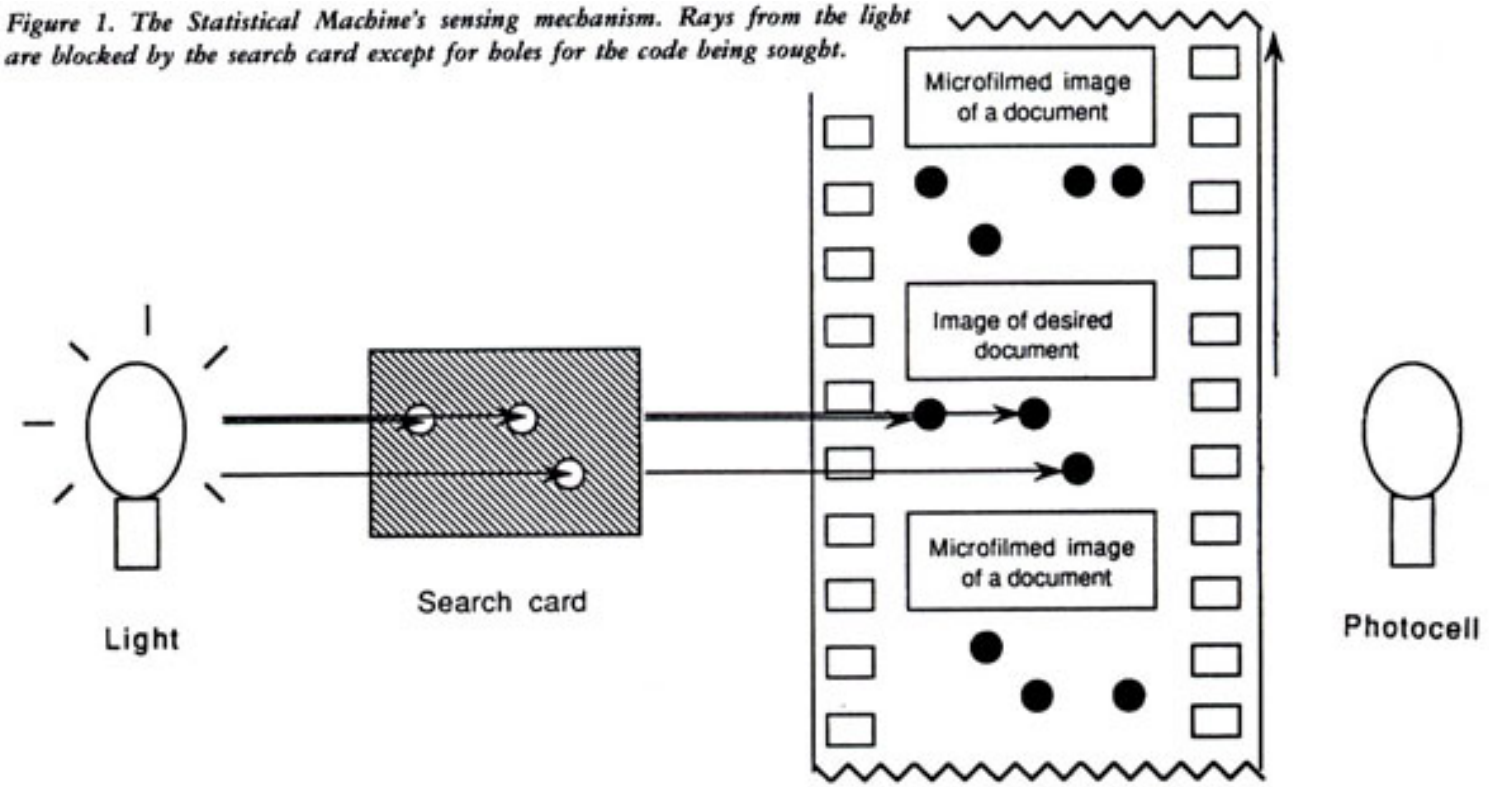
IR-related Tasks

- Ad-hoc retrieval (querying) – pull technology
- Information filtering and Recommendation – push technology
- Categorisation
- Clustering
- Browsing

Goldberg's Statistical Machine

A search engine[a] before there were computers (as we know them)!

Figure 1. The Statistical Machine's sensing mechanism. Rays from the light are blocked by the search card except for holes for the code being sought.



[a] https://people.ischool.berkeley.edu/~buckland/statistical.html

A Very Brief History of IR: 20s–70s

- **1920's**: Goldberg's Statistical Machine
- **1940's**: Vannevar Bush's "As We May Think"[1]
- **1950's**: First research efforts; Luhn, Bagley, Mooers allegedly coined the term "information retrieval"
- **1960's**: Cleverdon's Cranfield studies in IR evaluation; first IR book published by Becker and Hayes; Spärck Jones / Salton on the $tf \times idf$ weighting scheme; formulation of probabilistic models
- **1970's**: Jardine and van Rijsbergen on the cluster hypothesis; van Rijsbergen publishes his classic book entitled "Information Retrieval"[2]; first SIGIR conference

[1] http://www.theatlantic.com/doc/194507/bush
[2] http://www.dcs.gla.ac.uk/Keith/Preface.html

A Very Brief History of IR: 80s–20s

- **1980's**: Salton/McGill publish their classic book "Introduction to Modern Information Retrieval", which focussed on the Vector Space Model, hypertext retrieval, probabilistic models, information seeking
- **1990's**: Web Information Retrieval (Google), Digital Libraries, Topic Models (Latent Semantic Indexing), further probabilistic and logic-based models (Okapi BM25, Language Models)
- **2000's**: Learning to rank, interactive IR, cognitive models, semantic models, cross-lingual IR
- **2010's**: Interactive and session-based IR, semantic search, knowledge graphs, word embeddings (word2vec)
- **2020's**: Neural IR (deep learning), dense retrieval, embedding models (BERT), conversational agents, Large Language Models (LLMs) & generative AI, Retrieval-Augmented Generation (RAG)

Informal Definition

Information Retrieval is concerned with the

- Representation
- Storage
- Organisation and
- Access

of information (mainly documents).

Goal of an IR system: Retrieve all **relevant** documents while retrieving as few **non-relevant** documents as possible.

Documents in IR

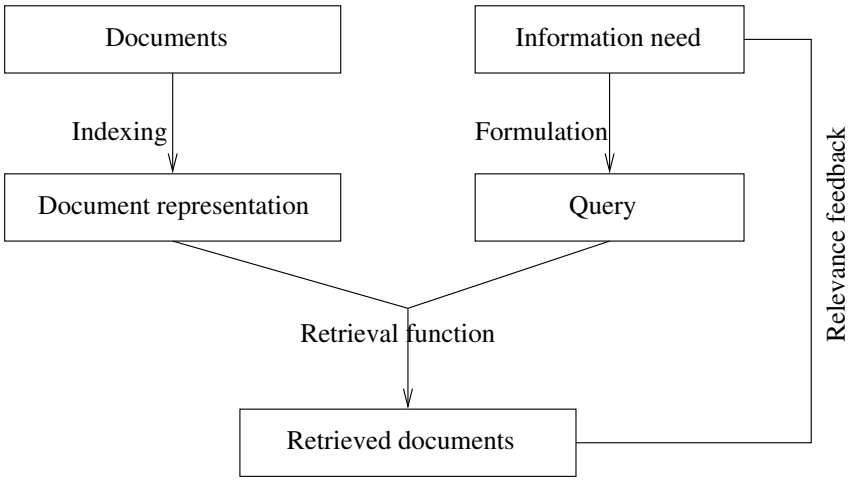
- Usually IR deals with (but is not limited to) retrieving documents
- By **document** we mean several kinds of information objects, for instance
  - Text documents (including Web pages)
  - Multimedia documents (sound, video, image)
- We mainly focus on text documents

IR Compared to Databases (DB)

- Formulation of the information need (as a natural query)
- Query returns many documents (see Google), but only a few are relevant
- Ranking: **best (IR) vs exact (DB) matching**
- Uncertainty of knowledge

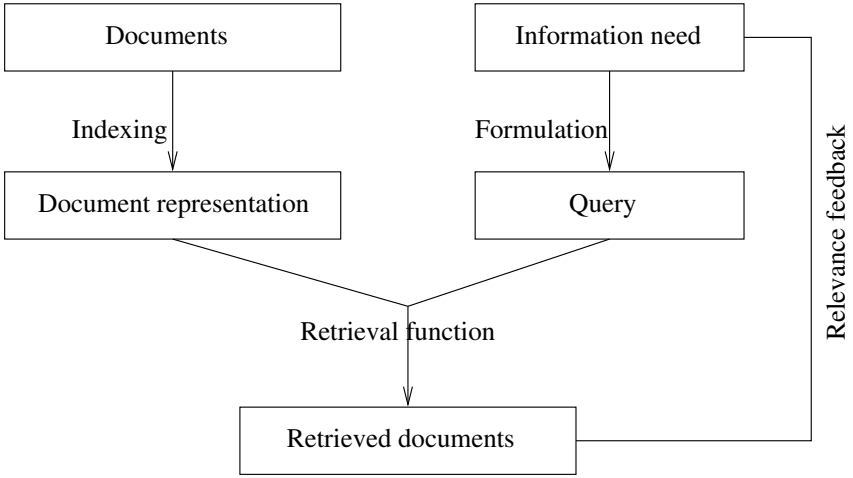
Conceptual Model

- Documents need to be transformed into **document representations** (**Indexing**)
- Users formulate a **query** from their **information need**
- Document representations and query are machine processable

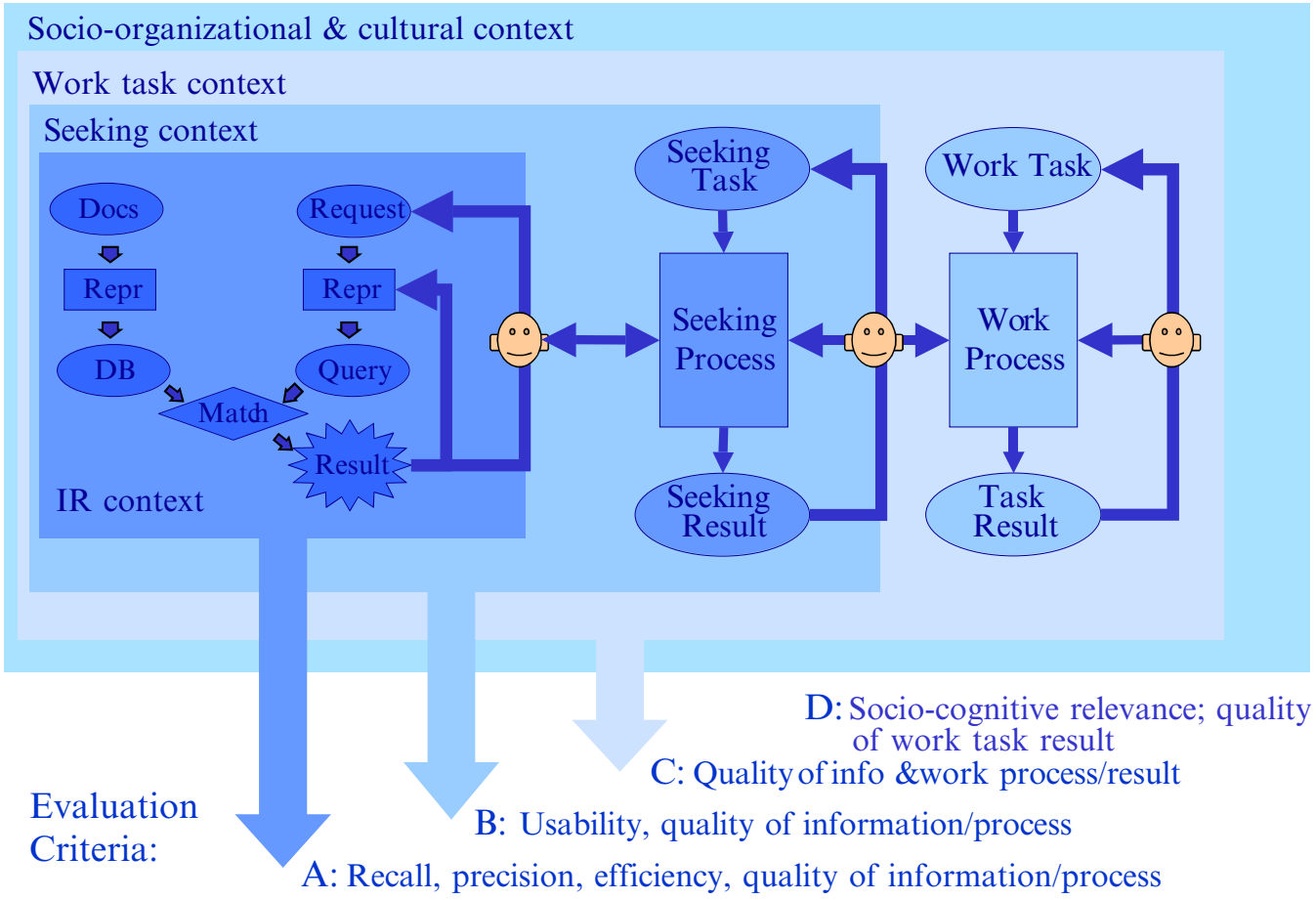


Conceptual Model

- A **retrieval function** creates a **ranked list of documents**
  - Usually by computing a **Retrieval Status Value** (a score)
- **Relevance Feedback** may be used to refine the search

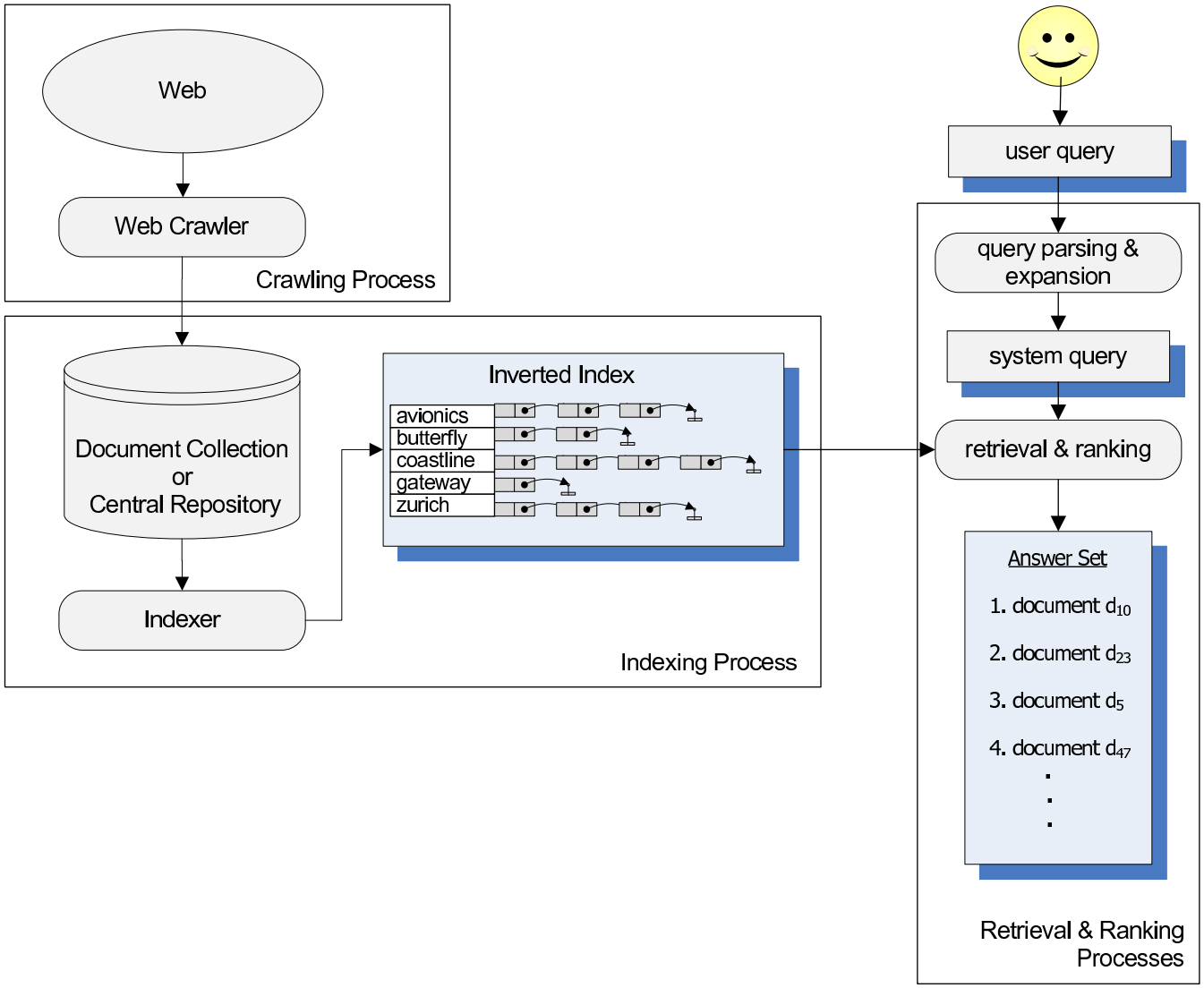


Task-based Information Search & Retrieval



Ingwersen/Järvelin: The Turn (p. 322)

Software Architecture of a (Web) Search Engine



Indexing

Indexing I

- The **index** is a data structure built from text to speed up the search
- Based on the index we can process queries and return documents
- Crucial step (for text documents): extract meaningful terms and record their occurrence in an **inverted index**
- **Terms** are strings of (possibly special) characters that are separated by
  - white spaces
  - punctuation and
  - other special characters

Indexing Process (4 Steps)

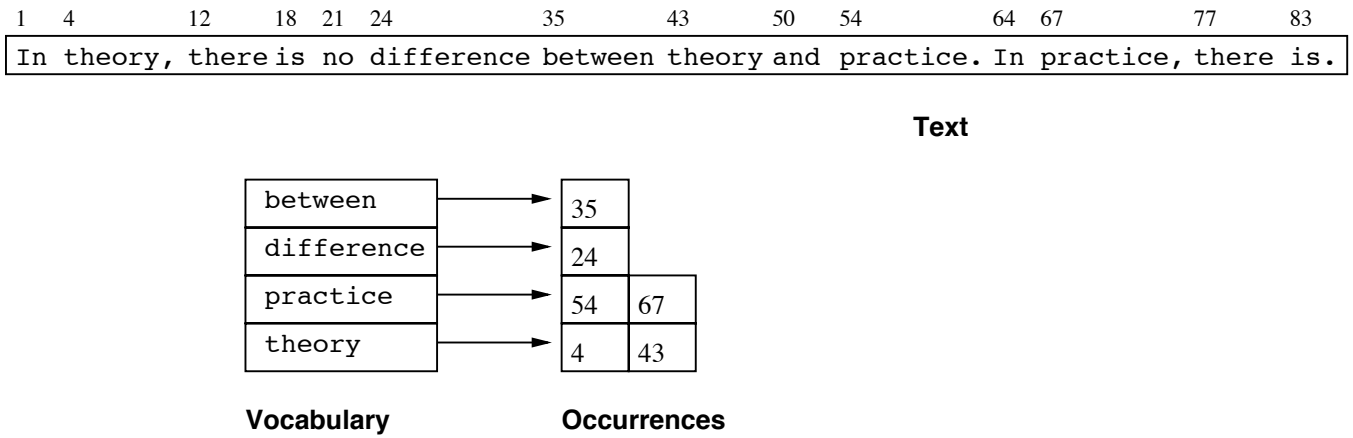
Indexing is usually performed in **4 steps**:

1. Tokenisation
2. Stopword elimination
3. Stemming
4. Creation of the inverted index

Indexing Step 1: Tokenisation

- **Tokenisation**: First of all we need to extract all **terms** in a document (we do this for all documents in the collection)
-
  - A term might be a single word, but could also be a phrase
  - For example, ‘computer science’ could be one term, or split into the two terms ‘computer’ and ‘science’
- We may not only consider the occurrence of a term, but also its position in the document
  - Useful for query term highlighting or phrase and proximity search (not further covered here)
- This builds our **vocabulary** (the set of all different terms found in all documents in the collection)

Tokenization Example



- Some terms are deleted (stopwords, next slide)
- Terms and their occurrences can be stored in a so-called inverted list (explained later)

Indexing Step 2: Stopword Elimination

- Frequent terms like “and”, “or” don’t bear any meaning for ad hoc retrieval
- This means we can eliminate these terms to keep our vocabulary small (usually stopword lists are applied)
- Example

  Apple has a built-in RSS reader. Kinda. While it’s not the dedicated RSS client that I hope they’ll build for iPhone, it helps a news junkie get his fix on a slow EDGE data connection.

Indexing Step 2: Stopword Elimination

- Frequent terms like “and”, “or” don’t bear any meaning for ad hoc retrieval
- This means we can eliminate these terms to keep our vocabulary small (usually stopword lists are applied)
- Example

  Apple has a built-in RSS reader. Kinda. While it’s not the dedicated RSS client that I hope they’ll build for iPhone, it helps a news junkie get his fix on a slow EDGE data connection.

  After stopword elimination (and converting terms to lower case):

  apple built-in rss reader kinda dedicated rss client hope build iphone helps news junkie get fix slow edge data connection

Indexing Step 3: Stemming

- **Stemming** is the process of reducing terms to their **stem base**
- More potentially relevant documents can be found
- Stemming method for the English language: **Porter algorithm**
- Several types of stemmers for different languages exist (for instance dictionary-based)
- Further Resources
  - Porter Stemmer Page
    `http://tartarus.org/~martin/PorterStemmer/`

Stemming Example

- Example:
  - `computer` $\longrightarrow$ `comput`
  - `computers` $\longrightarrow$ `comput`
  - `computing` $\longrightarrow$ `comput`

  $\rightarrow$ Search for 'computer' also finds documents with 'computers' or 'computing' (if we stem both the document and query terms)

Stemming Example

- Example:
  - `computer` $\longrightarrow$ `comput`
  - `computers` $\longrightarrow$ `comput`
  - `computing` $\longrightarrow$ `comput`

  $\rightarrow$ Search for 'computer' also finds documents with 'computers' or 'computing' (if we stem both the document and query terms)
- The previous text example after stopword elimination and stemming:

  appl built rss read kinda dedic rss cli hop build iphon help new junki get fix slow edg data connect

Stemming and Stopword Elimination

Both stemming and stopword elimination are applied to all documents, but also to each query.

Indexing Step 4: Inverted Index and Inverted List

- After tokenisation, stemming and stopword elimination (to create our vocabulary) we can build the inverted index
  - Ensures fast access to documents containing a term, similar to an index in a book
- The **inverted index** consists of inverted lists
- **Inverted list**: One of the basic data structures in IR
- Term-oriented
- There is an inverted list for each term in the vocabulary
- Each such list stores
  - The identifiers (IDs, e.g. URLs) of the documents that contain that term
  - All its positions in the document (if needed)
  - Further information as required
- We can now efficiently process queries and retrieve documents with the inverted index

Example Inverted Index

retrieval systems

DOCID	POS
d1	10,25, 37
d2	23
d4	45,55
d7	3,56

multimedia

DOCID	POS
d4	18,20
d6	22,27
d7	5
d9	3,96, 127

images

DOCID	POS
d1	14,68
d3	29,73, 235
d4	11,145
d7	10

Boolean Retrieval Model

Boolean Model

- First retrieval model
- No term weighting, only occurrence of a term in a document considered
- Exact match: strict separation into matching and non-matching documents (like in databases)
- INSPEC search is an example of Boolean retrieval
- Nowadays widely used for instance in prior art search for patent retrieval (large Boolean queries)

Searching in the INSPEC System

- INSPEC: INformation System for Physics, Electronics and Computing
- Documents, articles and books described by
  - bibliographic records
  - abstracts
  - categorisation into an hierarchical index system
  - keywords
- Boolean retrieval system: returns documents containing the keywords in the given form

INSPEC-Dokument

```
INSPEC XXXXXXXXX
Doc Type: Journal Paper
Title: Images database management system: a 'server-client producer' system on
       a local network and on the Internet
Authors: Ageron, P.; Besson, F.; Desfarges, P.
Affiliation: Univ. Lumiere, Lyon, France
Journal: Computer Networks and ISDN Systems
Vol: 26 Iss: suppl., no.2-3 p. S101-6
Date: 1994
Country of Publication: Netherlands
ISSN: 0169-7552 CODEN: CNISE9
CCC: 0169-7552/94/$07.00
Language: English
Treatment: Practical
Abstract: Lumiere University offers a full range of arts and social sciences.
The research resources are very large and are specialized in politics, economics,
finance, arts and archeology. As soon as the first network services were
available, researchers tried to find information on the network about digitised
photo management and everything about multimedia communication. The 'Images
Database Management System' program tries to manage all information about
digitised images  ...
Network like WAIS, WWW, or GOPHER and hope to place this tool at the Internet's
users' disposal. (0 Refs.)
```

Example Search: AND

Literature review on

"Retrieval systems for multimedia objects, in particular images"

Example Search: AND

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- We need a suitable query!
- Possible query:
  `retrieval system AND multimedia AND images`

Example Search: AND

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- We need a suitable query!
- Possible query:
  `retrieval system AND multimedia AND images`
- Interpretation in a Boolean retrieval system:
  *Return all documents in which **all** of these terms appear at least once*

Example Search: AND

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- We need a suitable query!
- Possible query:
  `retrieval system AND multimedia AND images`
- Interpretation in a Boolean retrieval system:
  *Return all documents in which **all** of these terms appear at least once*
- Example of a document returned:
  *"**Multimedia** information **retrieval system** using knowledge for recognition of **images**"*
  - Note "retrieval system" is regarded as one term in this example

Example Search: OR

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- Another possible query:
  `retrieval system OR multimedia OR images`
- Interpretation in a Boolean retrieval system:
  *Return all documents in which **any** of these terms appears at least once*

Example Search: OR

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- Another possible query:
  `retrieval system OR multimedia OR images`
- Interpretation in a Boolean retrieval system:
  *Return all documents in which **any** of these terms appears at least once*
- Example of a document returned:
  *"Recognition of **images** in medical applications"*

Effect of AND/OR

Query	#Docs retrieved
`retrieval systems AND multimedia AND images`	3
`retrieval AND multimedia`	148
`retrieval OR multimedia`	2559

- AND is a strict filter (less documents found)
- OR is less strict (many documents found)

Implementing Boolean Retrieval with Inverted Lists

- Each inverted list can be regarded as a set containing document IDs
  - Each document ID appears at most once in the inverted list of a term
- We can map Boolean operators to set operators applied on inverted lists
  - AND: Intersection
  - OR: Union
  - NOT: Minus

Another Boolean Search Example

Emergency Medicine Search

"Heart/physiopathology"[MAJR] OR ("Liver/pathology"[MAJR] OR "Liver/physiopathology"[MAJR]) OR ("Brain/pathology"[MAJR] OR "Brain/physiopathology"[MAJR]) OR ("Lung/pathology"[MAJR] OR "Lung/physiopathology"[MAJR]) OR ("Emergency Medical Services"[MAJR] OR "Wounds and Injuries"[MAJR] OR "Forensic Medicine"[MAJR] OR "Forensic Dentistry"[MAJR] OR "Mortuary Practice"[MAJR] OR "Homicide"[MAJR] OR "Suicide"[MAJR] OR "Crime"[MAJR] OR "Child Abuse"[MAJR] OR "Sex Offenses"[MAJR] OR "Drowning"[MAJR] OR "Accident Prevention"[MAJR] OR "Accidents, Occupational"[MAJR] OR "Bereavement"[MAJR] OR "Postmortem Changes"[MAJR] OR "Death, Sudden"[MAJR] OR "Fires"[MAJR] OR "Disasters"[MAJR] OR "Cardiopulmonary Resuscitation"[MAJR] OR "Autopsy"[MAJR]) OR ("Ethanol"[MAJR] AND "humans"[MeSH Terms]) OR ("Poisoning"[MAJR] AND "humans"[MeSH Terms]) OR ("pathology"[Subheading] AND ("Heart"[MAJR] OR "Brain"[MAJR] OR "Liver"[MAJR] OR "Lung"[MAJR]) AND Review[ptyp]) OR ("mortality"[Subheading]) OR "Death"[MeSH] OR ("Mortality/drug effects"[MeSH] OR "Mortality/pathology"[MeSH] OR "Mortality/trends"[MeSH]) OR ("Emergency Medical Services"[MeSH:noexp] OR "First Aid"[MeSH] OR "Wounds and Injuries"[MeSH:noexp] OR "Forensic Medicine"[MeSH:noexp] OR "Forensic Dentistry"[MeSH] OR "Mortuary Practice"[MeSH:noexp] OR "Cause of Death"[MeSH] OR "Homicide"[MeSH:noexp] OR "Suicide"[MeSH:noexp] OR "Infanticide"[MeSH] OR "Crime"[MeSH:noexp] OR "Child Abuse"[MeSH:noexp] OR "Elder Abuse"[MeSH] OR "Sex Offenses"[MeSH:noexp] OR "Abortion, Criminal"[MeSH] OR "Drowning"[MeSH:noexp] OR "Accident Prevention"[MeSH:noexp] OR "Accidental Falls"[MeSH] OR "Accidents, Aviation"[MeSH] OR "Accidents, Home"[MeSH] OR "Accidents, Occupational"[MeSH:noexp] OR "Accidents, Traffic"[MeSH]) OR ("Liability, Legal"[MeSH] OR "Bereavement"[MeSH:noexp] OR "Firearms"[MeSH] OR "Wounds, Gunshot"[MeSH] OR "Death Certificates"[MeSH] OR "Postmortem Changes"[MeSH:noexp] OR "Death, Sudden"[MeSH:noexp] OR "Fires"[MeSH:noexp] OR "Iatrogenic Disease"[MeSH] OR "Disasters"[MeSH:noexp] OR "Sudden Infant Death"[MeSH] OR "Carbon Monoxide"[MeSH] OR "Electric Injuries"[MeSH:noexp]) OR ("Embolism, Air"[MeSH] OR "Anaphylaxis"[MeSH] OR "Asphyxia"[MeSH] OR"Wounds, Nonpenetrating"[MeSH:noexp] OR "Coroners and Medical Examiners"[MeSH] OR "Embolism, Fat"[MeSH] OR "Wounds, Penetrating"[MeSH:noexp] OR "Wounds, Stab"[MeSH:noexp] OR "Heat Exhaustion"[MeSH:noexp] OR "Rigor Mortis"[MeSH] OR "Cardiopulmonary Resuscitation"[MeSH:noexp] OR "Traumatology"[MeSH] OR "Autopsy"[MAJR]) OR ("Ethanol"[MeSH:noexp] AND "humans"[MeSH Terms]) OR ("Poisoning"[MAJR:noexp] AND "humans"[MeSH Terms]) OR ("police"[MeSH Terms] OR police[Text Word]) OR ("Substance Abuse, Intravenous"[MeSH] AND "pathology"[Subheading]) OR("Burns"[MeSH:noexp] AND "pathology"[Subheading]) OR ("Burns"[MeSH:noexp] AND "mortality"[Subheading]) OR ("Athletic Injuries"[MeSH] AND "mortality"[Subheading]) OR ("Hypothermia"[MeSH] AND "pathology"[Subheading] AND "humans"[MeSH Terms]) OR ("pathology"[Subheading] AND ("Heart"[MeSH:noexp] OR "Brain"[MeSH:noexp] OR "Liver"[MeSH:noexp] OR "Lung"[MeSH:noexp]) AND Review[ptyp]) AND English[lang]

(Thanks to Andrew MacFarlane for the example)

Boolean Retrieval with Inverted Lists: Example

multimedia AND images: d4, d7

retrieval systems

DOCID	POS
d1	10,25, 37
d2	23
d4	45,55
d7	3,56

multimedia

DOCID	POS
d4	18,20
d6	22,27
d7	5
d9	3,96, 127

images

DOCID	POS
d1	14,68
d3	29,73, 235
d4	11,145
d7	10

Boolean Retrieval with Inverted Lists: Example

multimedia OR images: d1, d3, d4
d6, d7, d9

retrieval systems

DOCID	POS
d1	10,25, 37
d2	23
d4	45,55
d7	3,56

multimedia

DOCID	POS
d4	18,20
d6	22,27
d7	5
d9	3,96, 127

images

DOCID	POS
d1	14,68
d3	29,73, 235
d4	11,145
d7	10

Boolean Retrieval with Inverted Lists

- Without an inverted list, how would we process a query like `multimedia AND images`?

Boolean Retrieval with Inverted Lists

- Without an inverted list, how would we process a query like `multimedia AND images`?
- We would have to scan all documents in our collection for the occurrence of both query terms – very time consuming!

Boolean Retrieval: Critical Reflection

+ Powerful language to formulate the information need as query (very expressive)

Boolean Retrieval: Critical Reflection

+ Powerful language to formulate the information need as query (very expressive)
– Query formulation as Boolean expression too difficult for unexperienced users (more for expert users)
– Size of the answer set difficult to control
– No ranking of documents (but can be combined with ranking mechanisms)
– Strict separation (works more like a filter, only **exact match**, near misses are cut off, but they can be relevant, too!)

Vector Space Model (VSM)

Term Space

- We have seen how we can build our vocabulary by means of term extraction, stemming and stopword elimination
- Based on the vocabulary we can create the **term space**
- This is a vector space where each dimension represents one term in our vocabulary
- If we have n terms in our collection, we get an n-dimensional term or vector space
- Each document and each query is represented by a vector in the term space

Formal Description

- Set of terms in our vocabulary: $T = \{t_1, \ldots, t_n\}$
- T spans an ***n*-dimensional vector space**
- Document d_j is represented by a **vector of document term weights**
- Query q is represented by a **vector of query term weights**

Document Vector

- Document d_j is represented by a **vector of document term weights** $d_{j_i} \in \mathbb{R}$:

$$\vec{d}_j = \begin{pmatrix} d_{j_1} \\ d_{j_2} \\ \dots \\ d_{j_n} \end{pmatrix}$$

- Document term weights can be computed using *tf* and *idf* (see below)

Document Vector

- Document d_j is represented by a **vector of document term weights** $d_{j_i} \in \mathbb{R}$:

$$\vec{d}_j = \begin{pmatrix} d_{j_1} \\ \boxed{d_{j_2}} \\ \dots \\ d_{j_n} \end{pmatrix}$$

Weight of term t_2 in document d_j

- Document term weights can be computed using *tf* and *idf* (see below)

Query Vector

- Like documents, a query q is represented by a **vector of query term weights** $q_i \in \mathbb{R}$:

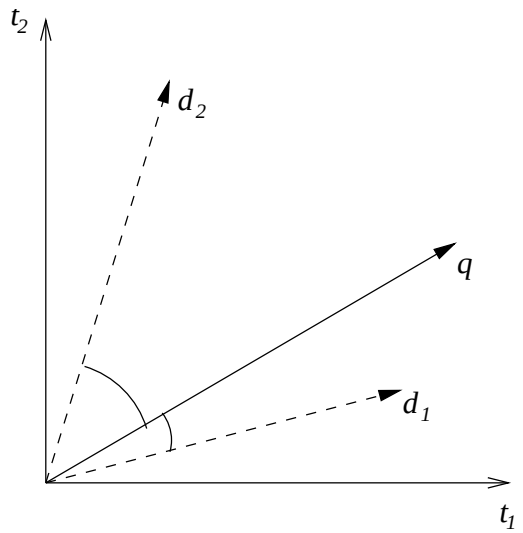
$$\vec{q} = \begin{pmatrix} q_1 \\ q_2 \\ \dots \\ q_n \end{pmatrix}$$

- q_i denotes the query term weight of term t_i
- q_i is 0 if the term does not appear in the query.
- q_i may be set to 1 if the term does appear in the query.
- Further query term weights are possible, for example
  - 2 if the term is important
  - 1 if the term is just “nice to have”

Retrieval Function

- The **retrieval function** computes a **retrieval status value (RSV)** using a vector similarity measure, e.g. the scalar product:

$$RSV(d_j, q) = \vec{d}_j \times \vec{q} = \sum_{i=1}^{n} d_{j_i} \times q_i$$



- **Ranking** of documents according to decreasing RSV

Example Query

Query: *"**side effects** of **drugs** on memory and cognitive abilities"*

t_i	Query $\vec{q}$	$\vec{d_1}$	$\vec{d_2}$	$\vec{d_3}$	$\vec{d_4}$
side effect	2	1	0.5	1	1
drug	2	1	1	1	1
memory	1	1	0	1	0
cognitive ability	1	0	1	1	0.5
RSV		5	4	6	4.5

Produces the ranking $d_3 > d_1 > d_4 > d_2$

Example Query

Query: *"**side effects** of **drugs** on memory and cognitive abilities"*

$$
\begin{aligned}
2 \times 1 &= 2 \\
2 \times 1 &= 2 \\
1 \times 1 &= 1 \\
1 \times 0 &= 0 \\
\hline
& \ 5
\end{aligned}
$$

t_i	Query $\vec{q}$	$\vec{d_1}$	$\vec{d_2}$	$\vec{d_3}$	$\vec{d_4}$
side effect	2	1	0.5	1	1
drug	2	1	1	1	1
memory	1	1	0	1	0
cognitive ability	1	0	1	1	0.5
RSV		5	4	6	4.5

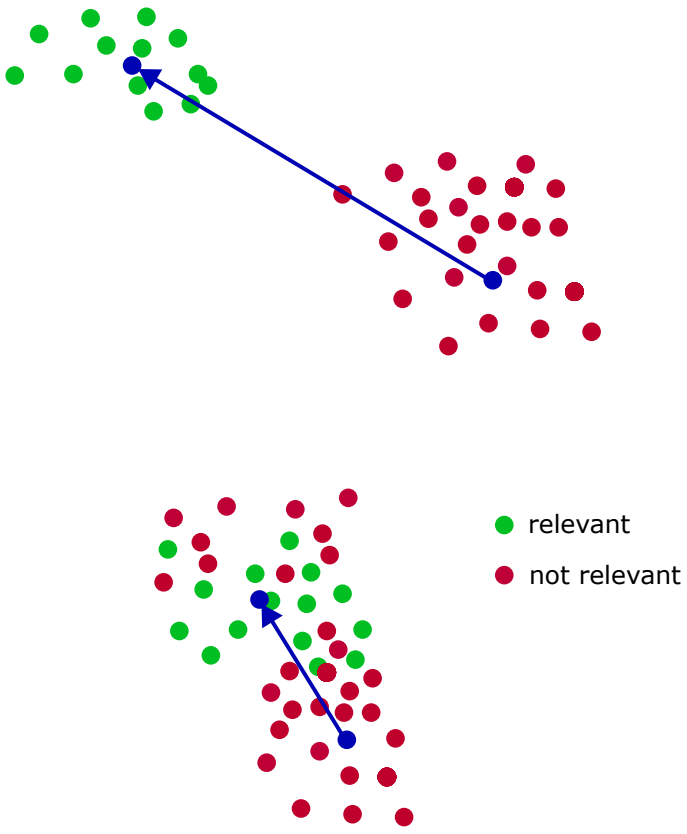
Produces the ranking $d_3 > d_1 > d_4 > d_2$

Relevance Feedback

- The knowledge of which documents are relevant and not relevant with respect to the query can be used by the retrieval system
- Experiments have shown this approach to be very effective
- Thought experiment: have you ever modified your query based on a relevant document in your search engine result page?
- This is the intuition behind relevance feedback – it helps getting better query terms

Relevance Feedback in the Vector Space Model

- Retrieve documents
- Specify which documents are relevant or not relevant with respect to the query (**relevance feedback**, **relevance judgements**)
- Move the query vector towards the centroid of the relevant documents
- Divide relevant from non-relevant documents
- Create a query vector that maximised the difference in RSVs between relevant and non-relevant documents



Rocchio Algorithm

1. Retrieve with original query vector $\vec{q}$
2. Relevance judgements for the top k documents in the ranking
3. Compute modified query vector $\vec{q}'$ with Rocchio formula
4. Retrieve with $\vec{q}'$
5. Potentially go to step 2

Rocchio Formula

$$\vec{q}' = \vec{q} + \alpha \frac{1}{|D^R|} \sum_{d_i \in D^R} \vec{d}_i - \beta \frac{1}{|D^N|} \sum_{d_j \in D^N} \vec{d}_j$$

$|D^R|$ number of relevant documents, $|D^N|$ number of non-relevant documents, α, β non-negative constants (e.g., $\alpha = 0.75, \beta = 0.25, \alpha + \beta = 1$) to emphasise negative or positive feedback

Types of Relevance Feedback

- Explicit relevance feedback
  - Users provide direct feedback whether documents are relevant or not
  - Tedious; users might not be willing
- Implicit relevance feedback
  - Infer relevance from user interaction and behaviour
  - User clicks document, puts document in a basket, dwell time on page, etc
- Pseudo relevance feedback
  - Assume the first top k documents in a ranking are relevant
  - k can be 1, 3, 5, 10, etc.

Term weighting with $tf \times idf$

Term Weighting

- In the VSM we described documents and queries as vectors of term weights
- **Document term weights** serve a purpose; they should reflect to what degree a term is suitable to be a **descriptor** of the document it appears in – the higher the weight, the more important the term for the document
- Furthermore, each term can be assigned a document-independent weight that reflects how valuable the term is to **discriminate** between documents

But how can we determine these term weights?

Practical Test

Take the given text and identify 4 terms that you think describe the text best!

Also think about why you chose these 4 terms.

Determining Term Weights

- You probably all had your good reasons to choose the terms you chose
- These terms should get a high term weight
- Left on its own devices, the computer does what it can do best - it **counts** and calculates!
- Two assumptions:
  - The more frequently a term appears in the document, the more important it is for describing the document ($\rightsquigarrow$ *tf*)
  - If a term appears in fewer documents, it is more suitable for distinguishing between documents ($\rightsquigarrow$ *idf*)

The Term Frequency *tf*

- Basic idea: Count the number of occurrences $occ(t, d)$ of a term t in the document d
- Example (after stemming and stopword elimination):

  appl built rss read kinda dedic rss cli hop they build iphon help new junki get his fix slow edg data connect

  $occ('rss', d) = 2$
- Different ways to use this value
  - Normalised *tf*
  - Fractional *tf*

The Normalised Term Frequency

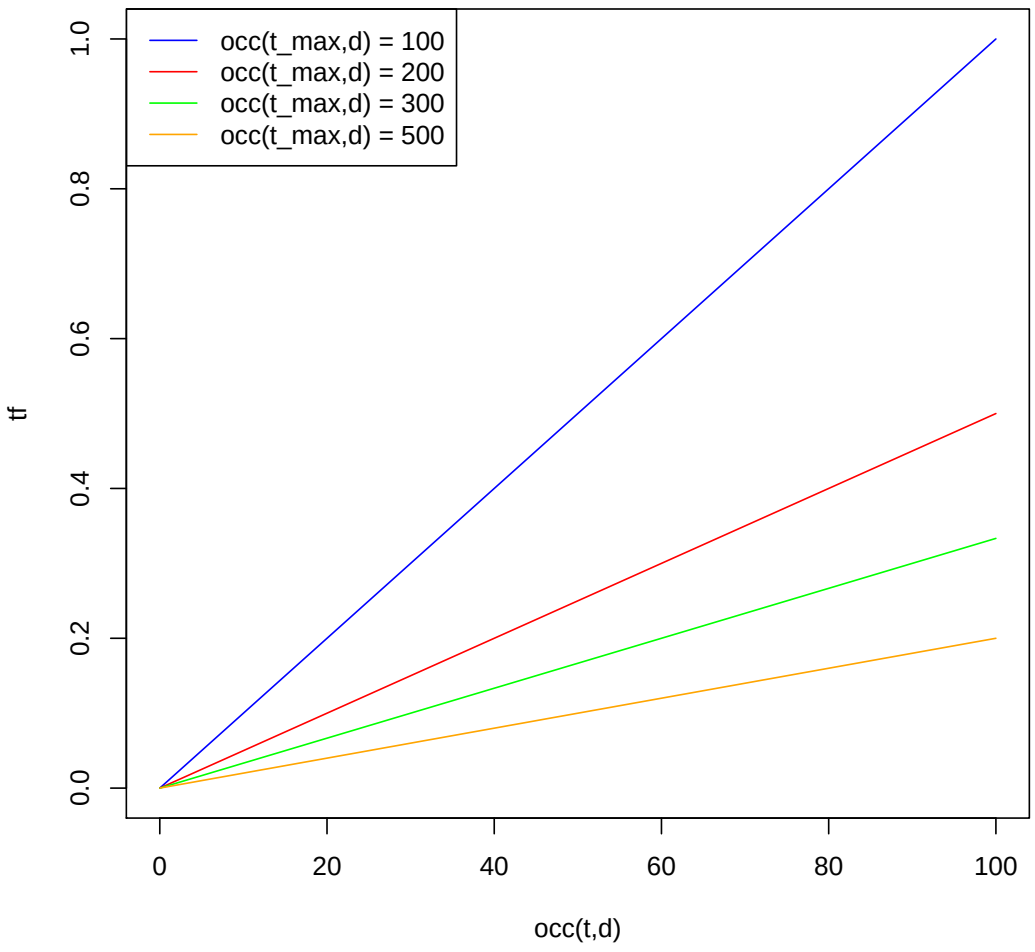
Normalised Term Frequency

$$tf(t,d) := \frac{occ(t,d)}{occ(t_{max},d)}$$

d	Document
t	Term in vocabulary
t_{max}	Most frequent term in document
$occ(t,d)$	Number of occurrences of t in d

- We divide (normalise) by $occ(t_{max},d)$ because otherwise larger documents would per se get higher term weights (compare Twitter tweets to whole books!)
- *tf* is 0 for terms not appearing in the document; *tf* is 1 for the most frequent term; all other terms get a *tf* between 0 and 1

Normalised Term Frequency



The Fractional Term Frequency

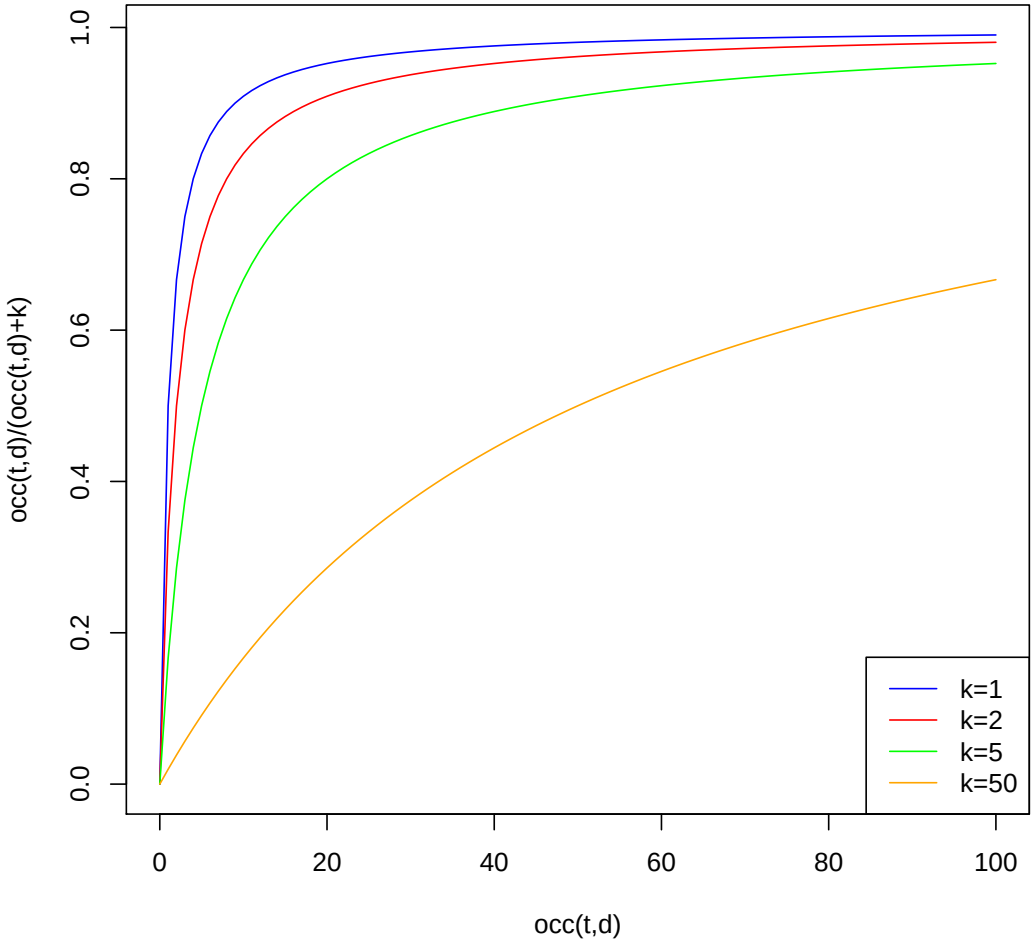
Fractional Term Frequency

$$tf(t,d) := \frac{occ(t,d)}{occ(t,d)+k}$$

d	Document
t	Term in vocabulary
k	Fractional parameter
$occ(t,d)$	Number of occurrences of t in d

- Delivers better and more stable retrieval quality
- Early occurrences of a term more important than later ones (steep curve, see next slide)
- k may depend on the document length (as used in BM25)

Fractional Term Frequency



The Inverse Document Frequency

Inverse Document Frequency

$$idf(t) := \log \frac{N}{n(t)}$$

t	Term in vocabulary
N	Number of documents in the collection
$n(t)$	Number of documents containing t

- High for rare terms (appearing only in few documents, good discriminators)
- Low for frequent terms (appearing in many documents)

idf variant

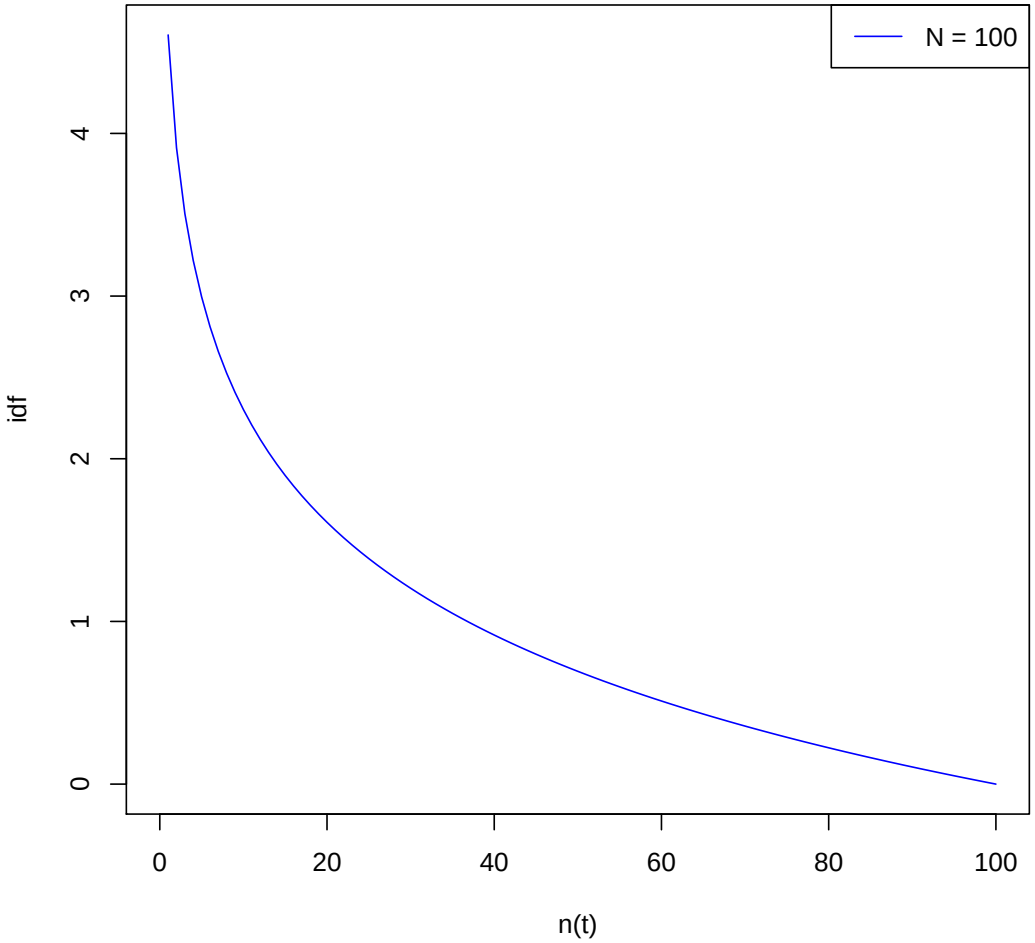
Inverse Document Frequency (smoothed)

$$idf(t) := \ln\left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1\right)$$

t	Term in vocabulary
N	Number of documents in the collection
$n(t)$	Number of documents containing t

- Derived from the Binary Independence Retrieval Model, a probabilistic model
- Other variants exist

Inverse Document Frequency



tf and *idf* in Document Vectors

- We can use *tf* and *idf* for the creation of document vectors in the term space (models using these term weights are usually called *tf* $\times$ *idf* models)
- Example:

$$\begin{aligned} \text{For } \vec{d}_j : d_{j_i} &= tf(t_i, d_j) \times idf(t_i) \\ \text{For } \vec{q} : q_i &= tf(t_i, q) \end{aligned}$$

- Then

$$RSV(d_j, q) = \sum_{t_i \in q} tf(t_i, q) \times tf(t_i, d_j) \times idf(t_i)$$

computes a Retrieval Status Value to rank documents with respect to the query

Okapi BM25

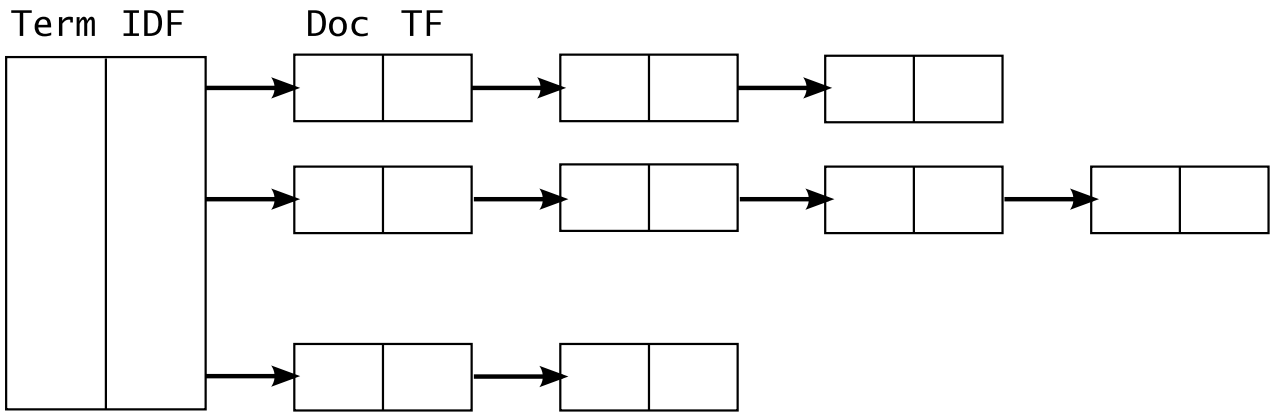
- The famous BM25 formula can be regarded as a variant of fractional term frequency with idf
- Wikipedia 2022 formulation: Given a query q containing keywords $q_1, \ldots, q_n$, the score for document d is:

$$RSV(d,q) = \sum_{i=1}^{n} idf(q_i) \times \frac{occ(q_i,d) \cdot (k_1+1)}{occ(q_i,d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{avgdl})}$$

- $|d|$ is the length of the document, *avgdl* the average document length in the collection
- k_1 and b are free parameters (often k_1 between 1.2 and 2, $b = 0.75$). We can ignore (k_1+1) (ranking invariant!)

Storing *tf* and *idf*

To store term weights, we can enrich inverted lists:



- Store the *idf* value together with the corresponding term
- Store the *tf* value together with the corresponding document entry in the term's inverted list

Vector Space Model: Critical Reflection

- **+** Easy and intuitive model
- **+** User friendly (no complex Boolean expression, just query terms needed)
- **+** Creates a ranking of documents (best match), no sharp cut-off as in the Boolean model (exact match)
- **+** Good results in combination with *tf* $\times$ *idf* or BM25 term weighting

Vector Space Model: Critical Reflection

- + Easy and intuitive model
- + User friendly (no complex Boolean expression, just query terms needed)
- + Creates a ranking of documents (best match), no sharp cut-off as in the Boolean model (exact match)
- + Good results in combination with $tf \times idf$ or BM25 term weighting
- – Less expressive than the Boolean model (user's got less means to specify information needs)
- – Extension of document representation difficult (for instance if title terms should be emphasised)

Further Models

Further Models – Overview

More models were developed in the last 40 years; some of them are

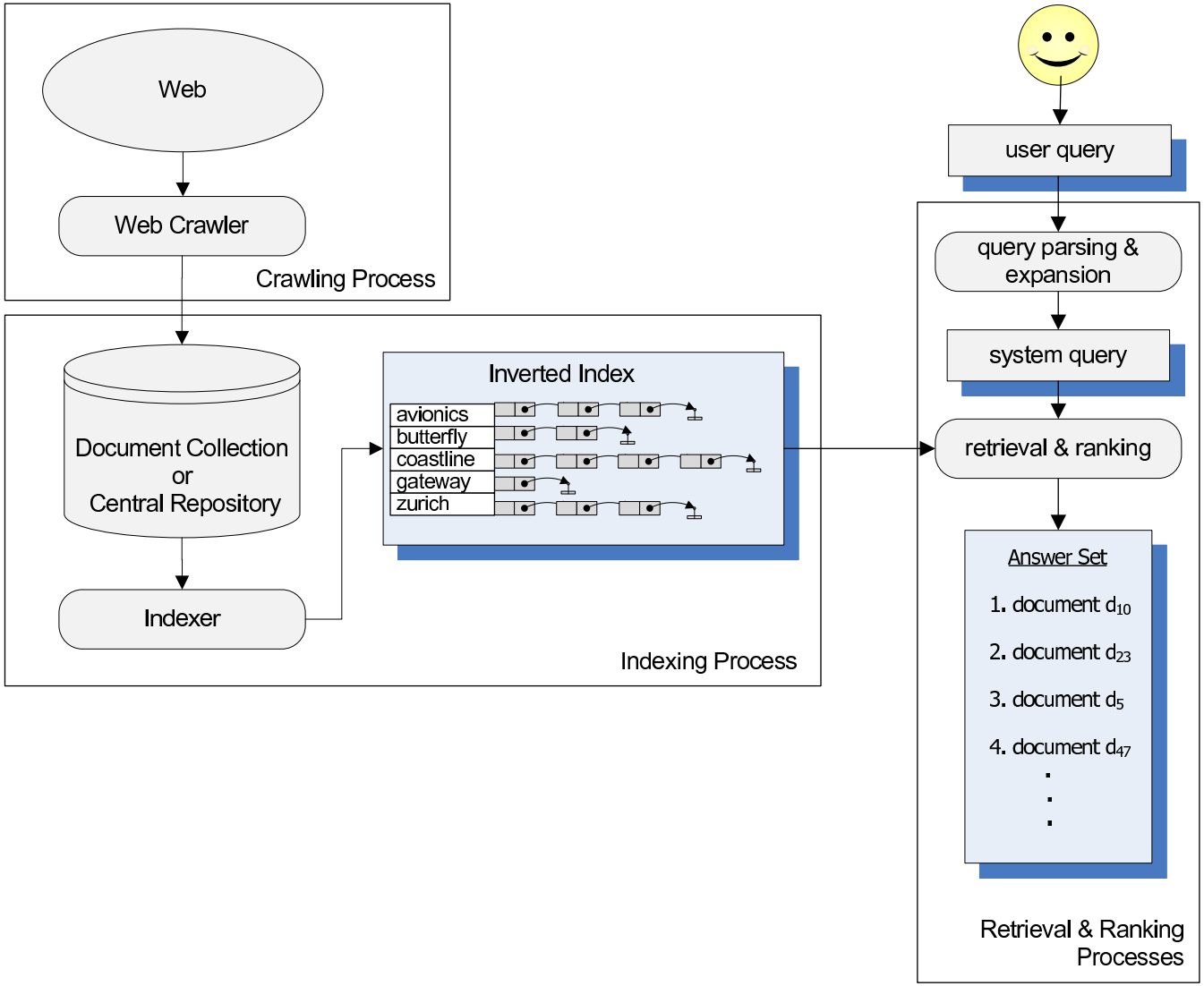
- Geometric models
  - Extended Vector Space Model
  - Word embeddings
  - Models inspired by Quantum Mechanics
- Probabilistic models
  - Language Models
  - BM25
  - Divergence from Randomness
  - Bayesian Networks
  - Models inspired by Quantum Mechanics
- Logic-based models
  - Fuzzy Model
  - Extended Boolean Model
  - Probabilistic Datalog

Further Models – Overview (cont'd)

- Semi-structured text, XML retrieval
- Passage retrieval
- Context-based retrieval (e.g., using user annotations)
- Discussion search
- Entity retrieval
- Multimedia and multimodal retrieval
- Interactive retrieval
- Retrieval-enhanced LLMs (Retrieval-Augmented Generation) and LLM-enhanced Retrieval
- And many many more...

Conclusion

Software Architecture of a (Web) IR System



Conclusion

- IR Overview: Conceptual Model and System Architecture
- Indexing: 4 steps (tokenisation, stopword elimination, stemming, inverted index)
- Boolean Model
- Vector Space Model: Term space, *tf*, *idf*, retrieval function, relevance feedback
- Further Models